Tartalomjegyzék

Kattints egy pontra, és a hozzá tartozó oldalra ugrik.

1. Bevezetés 2
1.1 A projekt célja 3
1.2 A megoldandó probléma 4
1.3 Célcsoport / felhasználók 5
1.4 A rendszer fő funkciói 6

2. Követelmények 7
2.1 Funkcionális követelmények 8
2.2 Nem funkcionális követelmények 9
2.3 Felhasználói szerepkörök 10

3. Rendszerterv 11
3.1 Architektúra 12
3.2 A rendszer komponensei 13
3.3 Kommunikáció a komponensek között 14
3.4 REST API 15
3.5 Alkalmazott technológiák 16

4. Adatbázis 17
4.1 Adatmodell 18
4.2 ER-diagram 19
4.3 Táblák részletes leírása 20
4.4 Kapcsolatok 21
4.5 Adatbázis létrehozása / dump 22

5. Felhasználói felület 23
5.1 Képernyők 24
5.2 Navigáció 25
5.3 Reszponzív működés 26
5.4 Felhasználói szerepkörök 27

6. Backend 28
6.1 Szerver felépítése 29
6.2 API végpontok 30
6.3 Hitelesítés / jogosultságkezelés 31
6.4 Hibakezelés 32

7. Frontend 33
7.1 Felépítés 34
7.2 Fontosabb funkciók 35
7.3 API használata 36

8. Telepítés és üzemeltetés 37
8.1 Szükséges környezet 38
8.2 Telepítés 39
8.3 Konfiguráció 40
8.4 Adatbázis létrehozása 41
8.5 Indítás 42

9. Tesztelés 43
9.1 Tesztelési stratégia 44
9.2 Tesztesetek 45
9.3 Automatizált tesztek 46
9.4 Teszteredmények 47
9.5 Hibák és javításuk 48

10. Forráskód és projektstruktúra 49
10.1 Repository 50
10.2 Könyvtárstruktúra 51
10.3 Fontosabb modulok 52

11. Csapatmunka 53
11.1 Csapattagok 54
11.2 Feladatmegosztás 55
11.3 Használt projektmenedzsment eszközök 56
11.4 Saját feladat / hozzájárulás 57

12. Összegzés 58

1. Bevezetés

1.1 A projekt célja

1.2 A megoldandó probléma

1.3 Célcsoport / felhasználók

1.4 A rendszer fő funkciói

1.1 A projekt célja

RÖVIDEN

A TutorConnect egy weboldal, ahol a diákok és a korrepetálók megtalálják egymást. A diák kiválasztja a tantárgyat, megnézi az oktatók árát és értékelését, aztán ír neki vagy foglal időpontot.

MIT SZERETNÉNK ELÉRNI?

- Hogy a diák gyorsan találjon oktatót abból a tantárgyból, amiből gyengébb.
- Hogy a korrepetáló egy helyen tudjon megjelenni, és észrevegyék.
- Hogy a kapcsolatfelvétel, az egyeztetés és a foglalás egy oldalon megoldható legyen.
- Hogy az értékelések alapján bátran lehessen választani.

KINEK MIT AD?

Diákoknak

Tantárgy és ár szerint kereshetnek, csevegnek az oktatóval, időpontot foglalnak, és az óra után értékelnek.

Korrepetálóknak

Megadják a tantárgyaikat és az óradíjukat, szabad időpontokat hirdetnek, a foglalásokat elfogadják vagy elutasítják.

Ehhez próbáljuk a lehető legtöbb eszközt létrehozni

Regisztráció, keresés, chat, foglalás és értékelés. Mind a két félnek, hogy gyorsabban létrejöjjön a kapcsolat.

1.2 A megoldandó probléma

MI A GOND MOST?

- A korrepetálók hirdetéseit alig veszik észre.
- A diákok nem tudják, hol keressenek korrepetáló tanárt.
- Nincs egy hely, ahol összehasonlítható az ár és a tantárgy.
- Nem látszik, hogy a korábbi diákok mit gondoltak az oktatóról.
- Az időpont egyeztetés hosszú, sokszor több üzenetváltás kell hozzá.

MIT AD ERRE A TUTORCONNECT?

Egy közös hely

Az oktatók egy listában vannak, tantárgyak és kategóriák szerint szűrve.

Átlátható ár

Minden oktatónál ott van az óradíj, tantárgyanként külön.

Értékelések

A diákok csillagokkal és szöveggel értékelnek az óra után.

Gyors egyeztetés

Chat és foglalási menü, ahol az oktató elfogadja vagy elutasítja a kérést.

Eredmény

Mind két fél gyorsabban talál egymásra, és kevesebb idő megy el a keresésre.

2. Követelmények

2.1 Funkcionális követelmények

2.2 Nem funkcionális követelmények

2.3 Felhasználói szerepkörök

3. Rendszerterv

3.1 Architektúra

3.2 A rendszer komponensei

3.3 Kommunikáció a komponensek között

3.4 REST API

3.5 Alkalmazott technológiák

3.3 Kommunikáció a komponensek között



A böngésző soha nem éri el közvetlenül az adatbázist, minden kérés a Main.js-en megy át.

HOGYAN KOMMUNIKÁLNAK?

REST API

A böngésző fetch kéréseket küld a Main.js-nek, az JSON-ban válaszol.

WebSocket

Új üzenetnél és foglalás változásnál a szerver azonnal szól a másik félnek.

Süti (munkamenet)

Minden kérés viszi a connect.sid sütit, így tudja a szerver, ki vagy.

Adatbázis

Csak a Main.js éri el a MySQL-t, a böngésző soha.

PÉLDA: ÜZENET KÜLDÉSE

1. **A diák elküldi az üzenetet**
   A chat.js POST kérést küld a szervernek.
2. **A Main.js ellenőrzi és elmenti**
   Megnézi, hogy be van-e jelentkezve, tagja-e a beszélgetésnek, aztán az adatbázisba írja.
3. **A másik fél azonnal megkapja**
   A szerver WebSocketen elküldi neki az új üzenetet.

3.4 REST API

HOGYAN MŰKÖDIK?

A böngésző és a szerver között a kommunikáció REST API-n megy. A böngésző HTTP kéréseket küld, az adatok JSON formában közlekednek. A végpontok erőforrások köré vannak szervezve, például /api/tutors, /api/bookings vagy /api/conversations.

HTTP METÓDUSOK

Metódus	Mire használjuk	Példa
GET	Adat lekérése	GET /api/tutors
POST	Új adat létrehozása	POST /api/bookings
PUT	Meglévő adat módosítása	PUT /api/account
PATCH	Részleges módosítás	PATCH /api/messages/:id
DELETE	Törlés	DELETE /api/availability/:id

VÁLASZKÓDOK

Kód	Mikor
200	Minden rendben, JSON válasz érkezik.
400	Hibás vagy hiányzó adat.
401	Nincs bejelentkezve.
403	Be van jelentkezve, de nincs joga hozzá.
404 / 409 / 500	Nem található / ütköző állapot / adatbázis hiba.

HIBAVÁLASZ FORMÁJA

```
{ "error": "Nem vagy bejelentkezve." }
```

4. Adatbázis

4.1 Adatmodell

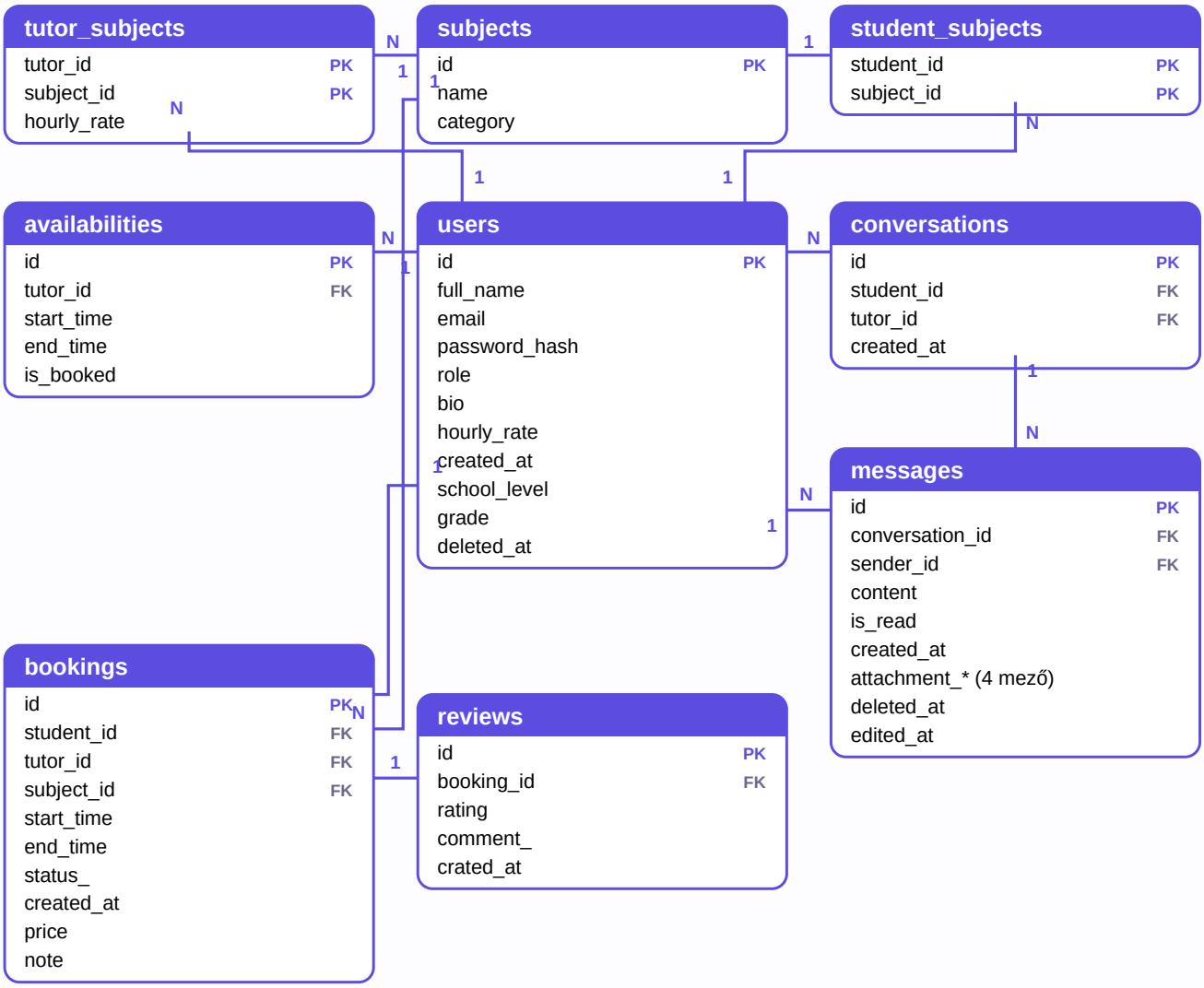
4.2 ER-diagram

4.3 Táblák részletes leírása

4.4 Kapcsolatok

4.5 Adatbázis létrehozása / dump

4.2 ER-diagram



PK	elsődleges kulcs
FK	idegen kulcs (másik tábla id-ja)
1 : N	egy a sokhoz kapcsolat

9 tábla

13 kapcsolat

Összekötő táblák

A tutor_subjects és a student_subjects kapcsolja a felhasználót a tantárgyhoz (N : M).

Anonimizálás

A törölt fiók a users táblában marad, a deleted_at oszlop jelzi.

4.4 Kapcsolatok

MELYIK TÁBLA KIHEZ KAPCSOLÓDIK?

Kapcsolat	Típus	Mi köti össze
users - availabilities	1 : N	Az oktató szabad időpontjai (tutor_id).
users - bookings	1 : N	A diák és az oktató is (student_id, tutor_id).
subjects - bookings	1 : N	Melyik tantárgyból van a foglalás.
bookings - reviews	1 : 1	Egy foglaláshoz egy értékelés.
users - conversations	1 : N	A diák és az oktató beszélgetése.
conversations - messages	1 : N	Egy beszélgetés üzenetei.
users - messages	1 : N	Az üzenet küldője (sender_id).
users - subjects	N : M	Összekötő tábla: tutor_subjects (oktató) és student_subjects (diák).

PÉLDA IDEGEN KULCSRA

```
ALTER TABLE `bookings`
  ADD CONSTRAINT `bookings_ibfk_1` FOREIGN KEY (`student_id`)
  REFERENCES `users` (`id`);
```

Törlés viselkedése

A conversations, messages és tutor_subjects kapcsolatai kaszkádolnak (ON DELETE CASCADE). A többi csak hivatkozási kényszer.

4.5 Adatbázis létrehozása / dump

ADATBÁZIS LÉTREHOZÁSA

1. **Indítsd el a MySQL-t**
   XAMPP Control Panelben a MySQL mellett nyomd meg a Start gombot.
2. **Hozz létre egy adatbázist**
   phpMyAdminban (http://localhost/phpmyadmin) készíts egy zsamo nevű adatbázist, utf8mb4_hungarian_ci egybevetéssel.
3. **Importáld a dumpot**
   Importálás fül, fájl: sql/zsamo.sql, majd Indítás.

PARANCSSORBÓL

```
mysql -u root -e "CREATE DATABASE zsamo"
mysql -u root zsamo < sql/zsamo.sql
```

A dump tartalma

Minden tábla szerkezete, a 42 tantárgy és néhány minta felhasználó, oktató, foglalás, üzenet.

Fontos

A minta felhasználók jelszava nem valódi hash, velük nem lehet belépni. Új fiókot regisztrálj az oldalon.

5. Felhasználói felület

5.1 Képernyők

5.2 Navigáció

5.3 Reszponzív működés

5.4 Felhasználói szerepkörök

5.1 Képernyők

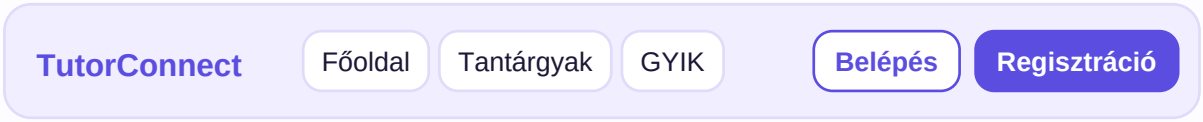
AZ OLDALAK CÍMEI

Oldal	Cím	Ki látja
Főoldal	/ vagy /index	mindenki
Tantárgyak	/subs	mindenki
Oktatók	/oktatok	mindenki
Chat	/chat	bejelentkezve
Foglalás	/foglalas, /foglalasutan	bejelentkezve
GYIK	/gyik	mindenki
Bejelentkezés	/login	mindenki
Regisztráció	/register	mindenki

5.2 Navigáció

A HÁROM MENÜ

Vendég (nincs bejelentkezve)



Diák



Korrepetáló



MIT JELENTENEK A MENÜPONTOK?

Főoldal, Tantárgyak

A nyitóoldal statisztikákkal, és a tantárgyak böngészése.

Oktatók, Chat

Az oktatók listája, és a beszélgetések (bejelentkezve).

Foglalás

Időpontok foglalása (diák) vagy megadása és elfogadása (korrepetáló).

GYIK

Gyakori kérdések, csak a vendég menüjében van.

MÉG A SÁVBAN

- Bejelentkezve megjelenik a profil menü a név kezdőbetűivel: fiókbeállítások és kijelentkezés.
- Van egy gomb, amivel világos és sötét téma között lehet váltani.
- A sávot a creator.js minden oldalon újra felépíti, a /api/me válasza alapján.

6. Backend

6.1 Szerver felépítése

6.2 API végpontok

6.3 Hitelesítés / jogosultságkezelés

6.4 Hibakezelés

6.1 Szerver felépítése

INDULÁS LÉPÉSRŐL LÉPÉSRE

1. **Express beállítása**
   JSON és űrlap adatok feldolgozása.
2. **Munkamenet**
   express-session: connect.sid süti, httpOnly, 7 nap. A titok a SESSION_SECRET változóból jön.
3. **Statikus fájlok**
   /css, /html, /images és a /js mappából csak a kliens fájlok. A main.js és az auth.js nem érhető el.
4. **Adatbázis kapcsolat**
   mysql2, localhost, root, adatbázis: zsamo.
5. **Végpontok és WebSocket**
   Az oldalak, az API és a chat kapcsolat regisztrálása ugyanazon a HTTP szerveren.
6. **Figyelés**
   A szerver a 3000-es porton indul el.

FONTOS ÉRTÉKEK

Mi	Érték
Port	3000
Süti neve és élettartama	connect.sid, 7 nap
Feltöltött fájl legnagyobb mérete	10 MB
WebSocket üzenet legnagyobb mérete	1 KB

> **Törölt fiókok**
> A törölt fiókok azonosítóit a szerver a memóriában tartja. Ha valaki még be van jelentkezve egy másik böngészőben, a következő kérésnél kijelentkeztetjük.

6.2 API végpontok

Végpont	Ki éri el	Leírás
GET /api/me	mindenki	Ki van bejelentkezve
POST /register, /login, /logout	mindenki	Regisztráció, belépés, kilépés
GET /api/subjects	mindenki	Tantárgyak listája
GET /api/tutors	mindenki	Oktatók listája szűréssel
GET /api/tutors/:id/reviews	mindenki	Egy oktató értékelései
GET /api/tutors/:id/availability	mindenki	Egy oktató szabad időpontjai
GET /api/stats	mindenki	Főoldali statisztikák
GET, POST /api/conversations	bejelentkezett	Beszélgetések, új beszélgetés
GET, POST /api/conversations/:id/messages	résztvevő	Üzenetek lekérése, küldése
POST /api/conversations/:id/attachments	résztvevő	Üzenet csatolmánnyal (max 10 MB)
POST /api/conversations/:id/read	résztvevő	Üzenetek olvasottnak jelölése
GET /api/messages/:id/attachment	résztvevő	Csatolmány megnyitása
PATCH, DELETE /api/messages/:id	küldő	Üzenet szerkesztése, visszavonása
GET, PUT, DELETE /api/account	bejelentkezett	Saját fiók, módosítás, törlés
GET, POST /api/availability	oktató	Szabad időpontok megadása
DELETE /api/availability/:id	oktató	Szabad időpont törlése
POST /api/bookings	diák	Foglalási kérelem
GET /api/bookings, /summary	bejelentkezett	Saját foglalások
POST /api/bookings/:id/confirm, /reject	oktató	Foglalás elfogadása, elutasítása
POST /api/bookings/:id/cancel	diák, oktató	Foglalás lemondása
POST /api/bookings/:id/review	diák	Értékelés leadása
GET /api/students/:id/profile	oktató	A beszélgetőpartner diák profilja

6.3 Hitelesítés / jogosultságkezelés

BEJELENTKEZÉS LÉPÉSEI

1. **Az adatok megérkeznek**
   A szerver megnézi, hogy nincs-e üres mező.
2. **A jelszó ellenőrzése**
   Az e-mail alapján megkeresi a felhasználót, és az Argon2 ellenőrzi a jelszót a hash-hez képest.
3. **A munkamenet létrejön**
   A szerver eltárolja az id-t, nevet, e-mailt és szerepkört. A böngésző csak az azonosítót kapja sütiben.

REGISZTRÁCIÓNÁL ELLENŐRIZZÜK

- Minden mező kötelező, a két jelszónak egyeznie kell.
- A név legfeljebb 60, az e-mail legfeljebb 100 karakter.
- A szerepkör csak diák vagy korrepetáló lehet, az e-mail egyedi.

JOGOSULTSÁG

Eset	Válasz
Nincs bejelentkezve	401 - Nem vagy bejelentkezve.
Rossz szerepkör (pl. diák időpontot hirdet)	403
Másik felhasználó beszélgetése vagy foglalása	404 / 403
WebSocket, törölt vagy hiányzó munkamenet	401, a kapcsolat megszakad

Fájlok védelme

A feltöltött fájl véletlen nevet kap, csak engedélyezett kiterjesztéssel (kép, pdf, Word, Excel, PowerPoint, txt) menthető, és csak a beszélgetés két tagja nyithatja meg.

6.4 Hibakezel s

MILYEN HIBA MIRE FIGYELMEZTET?

St tusz	Mikor	Mit l t a felhaszn l
400	res vagy hib s mez , t l hossz sz veg, nem enged lyezett f jl	Magyar hiba zenet, pl. "Minden mez t ki kell t lteni."
401	Nincs bejelentkezve	Nem vagy bejelentkezve.
403	Nincs joga a m velethez	Pl. "Csak di k foglalhat id pontot."
404	A keresett elem nem l tezik	Pl. "Az id pont nem tal lhat ."
409	Az llapot m r megv ltozott	Pl. a foglal s id k zben megv ltozott
500	Adatb zis hiba	ltal nos zenet, a r szleteket a szerver napl zza

Szerver oldalon

Az adatb zis hib kat a szerver ki rja a konzolra, de a felhaszn l nak csak egy r vid zenetet k ld. A szerver hiba ut n is tov bb fut.

Kliens oldalon

A fetch h v sok hib it az oldal elkapja, s ki rja a felhaszn l nak.

7. Frontend

7.1 Felépítés

7.2 Fontosabb funkciók

7.3 API használata

7.1 Felépítés

MAPPÁK

```
vsc/
  html/    az oldalak (index, subs, oktatok, chat, gyik ...)
  css/     stílusok, közös: style.css
  js/      oldalankénti szkriptek + creator.js
  images/  képek
```

MELYIK OLDAL MELYIK SZKRIPTET HASZNÁLJA?

Oldal	Szkriptek
index.html (főoldal)	creator.js, home.js
subs.html (tantárgyak)	creator.js, dropdown.js, index.js
oktatok.html	booking.js, oktatok.js, creator.js, selectmenu.js
chat.html	creator.js, booking.js, chat.js
foglalas.html	creator.js, foglalas.js
foglalasutan.html	creator.js, foglalasutan.js
login / register / gyik	creator.js

> **Közös rész**
> A creator.js minden oldalon fut: felépíti a menüt, a profil ablakot és a téma váltót.

7.3 API használata

HOGYAN HÍVJUK AZ API-T?

A kliens a fetch függvényt használja. A credentials: "include" miatt a böngésző elküldi a sütit is, így a szerver felismeri a felhasználót.

```
const response = await fetch("/api/me", {
    method: "GET",
    credentials: "include"
});
const data = await response.json();
```

MELYIK OLDAL MIT KÉR LE?

Szkript	Végpontok
creator.js	/api/me, /api/account, /api/subjects, /logout
home.js	/api/stats
index.js, oktatok.js	/api/subjects, /api/tutors
chat.js	/api/conversations, /api/messages, /api/students
booking.js	/api/bookings, /api/tutors
foglalas.js	/api/availability, /api/bookings
foglalasutan.js	/api/bookings, /api/bookings/summary

> **Hibák**
>
> Ha a szerver hibát küld, az error mező szövegét kiírjuk a felhasználónak.

8. Telepítés és üzemeltetés

8.1 Szükséges környezet

8.2 Telepítés

8.3 Konfiguráció

8.4 Adatbázis létrehozása

8.5 Indítás

8.1 Szükséges környezet

MI KELL A FUTTATÁSHOZ?

Program	Mire kell	Megjegyzés
Node.js és npm	A szerver futtatása	LTS verzió ajánlott
XAMPP (MySQL / MariaDB)	Az adatbázis	MariaDB 10.4 vagy újabb
Webböngésző	Az oldal megnyitása	Chrome, Firefox, Edge
Git	A kód letöltése	Vagy zip letöltés a GitHubról
Visual Studio Code	Szerkesztés	Nem kötelező

Argon2

Az argon2 csomag natív modul. Általában magától települ, de Windowson néha szükség van a build eszközökre.

Asztali alkalmazás

A C# programhoz Visual Studio és .NET 10 kell. A weboldalhoz ez nem szükséges.

Port

A 3000-es port legyen szabad.

8.2 Telepítés

LÉPÉSEK

1. **A projekt letöltése**
   Klónozd a repository-t, vagy töltsd le zip-ben.
2. **A függőségek telepítése**
   Windowson dupla kattintás az installer.bat fájlra. Vagy a mappában: npm install.
3. **Az adatbázis elkészítése**
   Lásd a 8.4 pontot.

```
git clone https://github.com/Matyi24/-TutorConnect-.git
cd -TutorConnect-
npm install
```

EZEKET A CSOMAGOKAT TELEPÍTI

Csomag	Mire használjuk
express	A webszerver
express-session	A bejelentkezés (munkamenet)
mysql2	Kapcsolat az adatbázissal
argon2	Jelszó hash-elés
multer	Fájlfeltöltés
ws	WebSocket (chat)

8.3 Konfiguráció

BEÁLLÍTÁSOK

Mit	Hol	Alapérték
Adatbázis kapcsolat	vsc/js/main.js	localhost, root, jelszó nélkül, zsamo
Munkamenet titok	SESSION_SECRET változó	fejlesztői alapérték
Teszt-naplózás	NODE_ENV, DEBUG_LOGS	fejlesztéskor be van kapcsolva
Port	main.js	3000
Süti secure	main.js	false (http-hez)

Éles használat előtt

Állítsd be a SESSION_SECRET változót, és használd a NODE_ENV=production beállítást. Fejlesztéskor a naplózás a jelszavakat is kiírja a konzolra!

HTTPS

Ha az oldal https-en fut, a munkamenet süti secure beállítását át kell írni true-ra.

Feltöltések

A chat fájlok a szerveren az uploads mappába kerülnek. Ez nincs verziókezelve.

8.4 Adatbázis létrehozása

ADATBÁZIS LÉTREHOZÁSA

1. **MySQL indítása**
   XAMPP Control Panel, MySQL, Start.
2. **Adatbázis létrehozása**
   phpMyAdmin: új adatbázis zsamo névvel, utf8mb4_hungarian_ci.
3. **Importálás**
   A sql/zsamo.sql fájl feltöltése az Importálás fülön.
4. **Ellenőrzés**
   A zsamo adatbázisban 9 tábla jelenik meg.

```
mysql -u root zsamo < sql/zsamo.sql
```

A 9 TÁBLA

users, subjects, tutor_subjects, student_subjects, availabilities, bookings, reviews, conversations, messages

Tábla	Mit tárol
users, subjects	Felhasználók és tantárgyak
tutor_subjects, student_subjects	Ki mit tanít, és ki miből kér segítséget
availabilities, bookings, reviews	Időpontok, foglalások, értékelések
conversations, messages	Beszélgetések és üzenetek

> **Séma kiegészítése**
> A szerver induláskor szükség esetén kiegészíti az adatbázist (pl. az üzenetek csatolmány oszlopait és az e-mail mező hosszát).

8.5 Indítás

INDÍTÁS

```
npm run dev            # újraindul, ha módosítod a fájlt
node vsc/js/main.js    # sima indítás
```

MIT LÁTSZ, HA JÓL MEGY?

```
TUTOR CONNECT SERVER STARTING
SERVER RUNNING
http://localhost:3000
```

1. **Nyisd meg az oldalt**
   A böngészőben a http://localhost:3000 címen.
2. **Regisztrálj**
   Hozz létre egy diák és egy korrepetáló fiókot.
3. **Próbáld ki**
   Keress oktatót, írj neki, foglalj időpontot.

> **Újraindítás**
> A bejelentkezések a szerver memóriájában vannak. Ha a szerver újraindul, mindenkinek újra be kell lépnie.

HA VALAMI NEM MŰKÖDIK

Hiba	Megoldás
ECONNREFUSED vagy adatbázis hiba	Nem fut a MySQL, indítsd el XAMPP-ban.
Unknown database zsamo	Hozd létre az adatbázist és importáld a dumpot.
EADDRINUSE	A 3000-es portot már használja valami, állítsd le.
Bejelentkezés után mindig kilép	A szerver újraindult, lépj be újra.

9. Tesztelés

9.1 Tesztelési stratégia

9.2 Tesztesetek

9.3 Automatizált tesztek

9.4 Teszteredmények

9.5 Hibák és javításuk

9.1 Tesztelési stratégia

HOGYAN TESZTELTÜNK?

Automatizált tesztek

A szerver végpontjait és a jelszó kezelést a Node.js beépített tesztfuttatója ellenőrzi.

Kézi tesztelés

Végigkattintottuk az oldalakat böngészőben, diákként, korrepetálóként és vendégként is.

Reszponzív próba

Megnéztük az oldalt telefonon és közepes monitoron.

AZ AUTOMATIZÁLT TESZTEK MŰKÖDÉSE

A tesztekhez nem kell valódi MySQL. A mysql2 modult egy memóriában lévő hamis adatbázis helyettesíti, a valódi szerver pedig véletlen porton indul el.



Futtatás

npm test

9.2 Tesztesetek

Teszteset	Várt eredmény
Regisztráció üres mezővel	400, hibaüzenet
Regisztráció eltérő jelszavakkal	400, "A két jelszó nem egyezik."
Bejelentkezés rossz jelszóval	Sikertelen belépés
Védett végpont bejelentkezés nélkül	401
Diák szabad időpontot hirdet	403
Idegen beszélgetés olvasása	Elutasítás
2000 karakternél hosszabb üzenet	Elutasítás
Nem engedélyezett fájl feltöltése	Elutasítás
Csatolmány elérése másik felhasználóval	404
Értékelés 1-5 csillagon kívül	400
Üzenet szerkesztése és visszavonása	Csak a küldő teheti
Fiók törlése	Az adatok anonimizálódnak, kijelentkezik
Név hosszabb mint 60 karakter	400
E-mail hosszabb mint 100 karakter	400
Regisztráció admin szerepkörrel	Elutasítás, csak diák vagy korrepetáló lehet
Túl hosszú értékelés szöveg	400

9.3 Automatizált tesztek

MIT FEDNEK LE A TESZTEK?

141	22	2
teszt	csoport	fájl

Csoport	Tesztelt terület
auth.test.js	Jelszó hash-elés és ellenőrzés (hashPassword, verifyPassword)
main.test.js: oldalak	Oldalak és statikus fájlok kiszolgálása
main.test.js: belépés	/api/me, regisztráció, bejelentkezés, kijelentkezés
main.test.js: adatok	Tantárgyak, oktatók, értékelések, statisztika
main.test.js: chat	Beszélgetések, üzenetek, szerkesztés, visszavonás
main.test.js: fájlok	Csatolmány feltöltése és letöltése
main.test.js: valós idő	WebSocket kapcsolat és értesítések
main.test.js: fiók	Fiók lekérése és módosítása

Futtatás

npm test. A tesztek helye: vsc/test (auth.test.js, main.test.js, helpers/server.js).

9.4 Teszteredmények

AZ UTOLSÓ FUTTATÁS

141	116	13	8
teszt összesen	sikeres	sikertelen	megszakadt



MI NEM MENT ÁT?

- Chat csatolmányok feltöltése és letöltése.
- Üzenet küldés és visszavonás néhány esete.
- A fiók adatainak lekérése és módosítása.
- Két tesztet TODO-nak jelöltünk: a feltöltött fájlok közvetlen elérése és a WebSocket hibakezelés túl nagy üzenetnél.

ÚJRAFUTTATÁS

```
cp -r vsc/test test
npm test
```

> **Megjegyzés**
>
> A tesztek a repository gyökerében lévő test mappából futnak, ezért a vsc/test mappát oda kell másolni, hogy az npm test megtalálja őket.

9.5 Hibák és javításuk

Hiba	Javítás
A szerver forráskódja elérhető volt a /js útvonalon	Fehérlistás kiszolgálás: csak a kliens fájlok érhetők el
A chat fájlok közvetlenül elérhetők voltak	Véletlen fájlnév, és csak a beszélgetés tagjai tölthetik le
Az oktatók e-mail címe kiszivároghatott	Az e-mail nem kerül ki az API válaszokba
Az ékezetes fájlnevek elromlottak (ő, ű)	A név újrakódolása UTF-8-ra, veszélyes karakterek törlése
Lejárt foglalások beragadtak	Lekérés előtt a lejártak automatikusan COMPLETED vagy CANCELLED állapotba kerülnek
Lemondás után az időpont foglalt maradt	Elutasításkor és lemondáskor az időpont újra szabad
A törölt fiók munkamenete tovább élt	A szerver kijelentkezteti, a WebSocket kapcsolata megszakad
Az e-mail mező túl rövid volt	Az adatbázis mező 100 karakteresre bővül, ha kell
A régi feltöltések a js mappában voltak	A szerver áthelyezi őket a feltöltések mappába
Túl nagy WebSocket üzenet leállíthatja a szervert	Még nincs javítva, a tesztekben TODO
Sötét mód, tantárgyak oldal kinézete	Újratervezés, egyedi legördülő menük

10. Forráskód és projektstruktúra

10.1 Repository

10.2 Könyvtárstruktúra

10.3 Fontosabb modulok

10.1 Repository

A REPOSITORY

GitHub

https://github.com/Matyi24/-TutorConnect-

80+	3	~12 600	main
commit	fejlesztő	sor JS kód	fő ág

HOGYAN DOLGOZTUNK?

- Mindenki a saját gépén fejlesztett, a változásokat GitHubra töltöttük fel.
- A más által feltöltött változásokat a main ágban egyesítettük (merge).
- A commit üzenetek leírják, mi változott (pl. sötét mód, biztonság, foglalás és értékelés).
- A kód nem tartalmazza a feltöltött fájlokat, azokat a .gitignore kihagyja.

HASZNOS PARANCSOK

```
git clone https://github.com/Matyi24/-TutorConnect-.git
git pull
git add .
git commit -m "üzenet"
git push
```

10.2 Könyvtárstruktúra

```
-TutorConnect-/
  installer.bat       függőségek telepítése
  package.json        csomagok és scriptek
  sql/
    zsamo.sql         adatbázis + minta adatok
  vsc/                a weboldal
    html/             oldalak
    css/              stílusok
    js/               main.js, auth.js + kliens
    images/           képek
    test/             automatizált tesztek
  Desktop/
    TutorConnectApp/  asztali alkalmazás váz (C#)
  dokumentácsó/       a dokumentáció
```

KÓD MÉRETE (SOR)

Fájl	Sor
vsc/js/main.js	kb. 3850
vsc/js/chat.js	kb. 2800
vsc/js/creator.js	kb. 1770
vsc/js/oktatok.js	kb. 1440
vsc/test/main.test.js	kb. 1930

Hol mi van?

A szerver a vsc/js mappában, a böngészőben futó rész a html, css és js mappákban van.

10.3 Fontosabb modulok

main.js

A teljes szerver: végpontok, munkamenet, WebSocket, feltöltés. Kb. 3800 sor.

auth.js

Jelszó hash-elés és ellenőrzés Argon2id-vel.

creator.js

Menü, profil menü, téma váltó, fiókbeállítások ablak.

chat.js

A chat oldal, WebSocket, csatolmányok.

oktatok.js

Oktatók listája, szűrés, értékelések.

booking.js, foglalas.js, foglalasutan.js

Az időpontfoglalás teljes folyamata.

dropdown.js, selectmenu.js

Egyedi legördülő menük a tantárgyak és az oktatók oldalán.

index.js, home.js

A tantárgyak oldal és a főoldal logikája.

test/helpers/server.js

A tesztkörnyezet: hamis adatbázis és szerverindítás.

Desktop/TutorConnectApp

Windows Forms váz: Form1.cs (ablak), NavBar.cs (oldalsáv), Theme.cs (színek).

11. Csapatmunka

11.1 Csapattagok

11.2 Feladatmegosztás

11.3 Használt projektmenedzsment eszközök

11.4 Saját feladat / hozzájárulás

11.1 Csapattagok

GitHub: Matt 26 commit

Profil oldal s fi kbe ll t sok
S t t m d, jelsz mutat sa a bel p sn l
Tesztek, telep t (installer.bat), dokument ci

GitHub: Aggrroo 27 commit

Chat val s id ben (WebSocket), f jl- s k pk ld s
Szerver, foglal s s rt kel s, biztons g
Anim ci k, oktat k s tant rgyak jratervez se

GitHub: Pelion0 15 commit

Oktat k s tant rgyak oldal, rt kel sek megjelen t se
CSS rendez s, bel p s s regisztr ci kin zete
Asztali alkalmaz s (C#), teend lista

11.2 Feladatmegosztás

KI MIVEL FOGLALKOZOTT?

Terület	Ki készítette
Szerver (main.js), API végpontok	Stickel Zsombor
Chat, WebSocket, csatolmányok	Stickel Zsombor
Foglalás és értékelés	Stickel Zsombor
Biztonság (fájlok, e-mail elrejtése)	Stickel Zsombor
Profil és fiókbeállítások	Porkoláb Mátyás Attila
Sötét mód	Porkoláb Mátyás Attila, majd Stickel Zsombor (minden oldalra)
Tesztek és telepítő	Porkoláb Mátyás Attila
Dokumentáció	Porkoláb Mátyás Attila
Oktatók és tantárgyak oldal	Marián Máté (később Stickel Zsombor újratervezte)
CSS, belépés és regisztráció kinézete	Marián Máté
Asztali alkalmazás (C#)	Marián Máté

COMMITOK SZÁMA

26	27	15
Porkoláb Mátyás (Matt)	Stickel Zsombor (Aggrroo)	Marián Máté (Pelion0)

Közösen

A tervezést, az ötleteket és a hibák keresését együtt csináltuk. A változásokat GitHubon egyesítettük.

11.3 Használt projektmenedzsment eszközök

MIRE HASZNÁLTUK?

Eszköz	Mire
GitHub	Közös repository, a változások követése, ágak egyesítése
Git	Verziókezelés
Visual Studio Code	A weboldal fejlesztése
Visual Studio	Az asztali alkalmazás váza
XAMPP, phpMyAdmin	Az adatbázis futtatása és kezelése
dbdiagram.io	Az ER-diagram elkészítése
npm_list fájl	Közös teendőlista a kódban (vsc/js/npm_list)

> **Munkafolyamat**
>
> Feltöltés GitHubra, egyesítés a main ágban, majd mindenki letölti a legfrissebbet.

11.4 Saját feladat / hozzájárulás

Porkoláb Mátyás Attila (GitHub: Matt)

Ő készítette a profil oldalt és a fiókbeállításokat, a sötét mód első részét, a jelszó mutatása gombot a belépésnél, a telepítőt (installer.bat) és az automatizált teszteket (141 teszt, hamis adatbázissal). Ő írta a dokumentációt is.
Technológiák: JavaScript, CSS, Node.js tesztfuttató.

Stickel Zsombor (GitHub: Aggrroo)

Ő készítette a chatet: valós idejű üzenetküldés WebSocketen, üzenet szerkesztése és visszavonása, kép és fájl küldése, képnézegető. Ő bővítette a szervert a foglalással, az értékeléssel és a profil adatokkal, és ő csinálta a biztonsági javításokat. A sötét módot minden oldalra kiterjesztette, újratervezte az oktatók és a tantárgyak oldalt.
Technológiák: Node.js, Express, ws, multer, MySQL, JavaScript.

Marián Máté (GitHub: Pelion0)

Ő készítette az oktatók és a tantárgyak oldal első változatát, javította az értékelések megjelenítését, és rendbe tette a CSS-t. Ő alakította ki a belépés és a regisztráció kinézetét, az alap stílust és a profil menü alapját. Ő hozta létre az asztali alkalmazás (C# Windows Forms) vázát és a közös teendőlistát.
Technológiák: HTML, CSS, JavaScript, C# (.NET).

12. Összegzés

A projekt egy mondatban

Egy weboldal, ahol a diákok és a korrepetálók gyorsan megtalálják egymást, egyeztetnek és időpontot foglalnak.

MIT ÉRTÜNK EL?

A TutorConnect célja az volt, hogy a diákok és a korrepetálók gyorsan egymásra találjanak. Ez megvalósult: lehet regisztrálni, keresni, csevegni, időpontot foglalni és értékelni.

9	30+	141	3
adatbázis tábla	API végpont	automatizált teszt	szerepkör

Ami működik

Regisztráció és belépés, tantárgy és oktató keresés, valós idejű chat fájlokkal, foglalás elfogadással, értékelés, sötét mód.

Amit tanultunk

A csapatmunka és a verziókezelés fontosságát, a biztonságot (jogosultságok, fájlkezelés), és hogy a tesztek sok hibát megfognak.

MERRE FEJLESZTHETNÉNK TOVÁBB?

- Tartós munkamenet tár, hogy újraindítás után is bejelentkezve maradjon mindenki.
- HTTPS és az adatbázis beállítások környezeti változókból.
- Online fizetés az órákhoz.
- Admin felület és az asztali alkalmazás befejezése.
- A hibás tesztek javítása.